

Model Fune-Tuning

COMP3110 Week3 - Kenneth Zhang

Why Model Fine-Tuning?

- **Domain adaptation.** Getting the model with specialised vocabulary and conventions, such as legal, medical, finance, or internal tasks.
- **Consistent output format or style.** Such as returning valid schema (JSON), writing in a brand voice, or matching a clinical-note structure.
- **Cost and latency.** You can fine-tune a small, cheap model to match a large model on your specific task. This is often done by distilling the big model's outputs.

What are the types of Model Fine-Tuning?

- **Supervised Fine-Tuning (SFT)**
 - You show the model examples of the behaviour you want and train it to imitate them. The data is (prompt, response) pairs. The loss is done on next-token prediction, usually computed only on the response tokens, so the model learns to produce the answer rather than predict the question.
- **Direct Preference Optimization (DPO)**
 - The model is shown two answers to the same question, one good and one bad, and learns to make the good answer more likely and the bad answer less likely. It measures this against a frozen copy of the original model, so it improves without drifting too far or losing its general ability.

What resources do we need for Model Fine-Tuning?

- Hardware - GPU memory (VRAM)



Model Setting

Model Name	Llama-3-8B-Instruct
Provider	Meta
Quantization	BF16
Attention Structure	GQA
Context Length	8192

Hardware Runs

GPU Model	NVIDIA A100-SXM4-80GB
GPU Count	1
Interconnect	PCIe Gen 4
Batch Size	4
Concurrent Users	1
KV Cache Precision	FP16

VRAM Allocation Breakdown

67%		20%	7%	6%
Component Item	Required Memory (GB)	Memory Proportion	Visual Color	
Base Model Weights	15.00 GB	66.8%		
KV Cache	4.50 GB	20.0%		
Activations (Estimate)	1.50 GB	6.7%		
Framework Overhead	1.45 GB	6.5%		

How to build Model Fine-Tuning environment?



uv is a single, very fast Python version managers. It installs the right Python version, creates the environment, installs packages, and locks exact versions.

Conda is a package and environment manager that installs both Python packages and non-Python system libraries (like CUDA or C++ libraries), so each project gets its own isolated setup.

Let's see some Model Fine-Tuning scripts



Supervised Fine-Tuning



Direct Preference Optimization (DPO)

